SQL Agent

A multi-model system for natural-language data analysis

AUTHORS

Daniel Regalado Cardoso, Nefeli Zafeiri, Oliver Mazariegos, Eleniz Espina

MSBA · University of Miami · 2026

PROBLEM STATEMENT

Tabular data analysis still requires SQL knowledge

Business users frequently have:

- Tabular data they need to query
- Specific questions in mind
- No working SQL knowledge

Existing options (manual SQL, generic chatbots, BI tools) each have trade-offs in cost, accuracy, or accessibility.

Objective

Build a system that:

1. Accepts a CSV or JSON file as input
2. Accepts a question in natural language
3. Returns the answer as a chart and a written finding
4. Runs at low cost on free GPU infrastructure

METHODOLOGY

Six phases from raw data to deployed system

Each phase produces a reproducible artifact: a dataset, a model adapter, or a deployable component. The repo contains the scripts to recreate each step from scratch.

PHASE 1 — SOURCING

Aggregating ten public text-to-SQL datasets

We selected ten existing datasets covering different schemas and SQL dialects:

- `b-mc2/sql-create-context`
- `gretelai/synthetic_text_to_sql`
- `knowrohit07/know_sql`
- `NumbersStation/NSText2SQL`
- `Clinton/Text-to-sql-v1`
- `motherduckdb/duckdb-text2sql-25k`
- `bugdaryan/spider-natsql-wikisql`
- `ChrisHayduk/Llama-2-SQL`
- `kaxap/llama2-sql-instruct`
- `PipableAI/spider-bird`

Rationale

Building a corpus from scratch was not feasible within the project timeline. Public datasets give us schema diversity and large coverage with permissive licenses.

Combined size

Approximately 1.2 million rows before cleaning. Heterogeneous formats and varying quality, requiring substantial pre-processing.

PHASE 2 — CURATION

Cleaning the merged corpus

1.2M	**761k**	**93%**	**723k**
Raw rows	After deduplication	Pass length filter	Final training set

All transformations are implemented as UV scripts in `training/data_pipelines/` and are reproducible end to end.

PHASE 3 — BUILDING TASK DATASETS

Three datasets, one per model task

Dataset	Rows	Source	Hub
text-to-sql-mix-v2	761,155	10 merged public sources	link
chart-reasoning-mix-v1	~75,000	nvBench (25k) + GPT-4.1-nano knowledge distillation (50k)	link
svg-chart-render-v1	~25,000	nvBench charts re-rendered via matplotlib SVG backend	link

All three are published openly on Hugging Face Hub under permissive licenses (Apache-2.0 and CC-BY-4.0).

QLoRA via Unsloth

We use 4-bit QLoRA with the Unsloth library for all three fine-tunes.

Reasons for this choice:

- Allows training a 7B model on a single 48 GB GPU
- Approximately 2× faster than vanilla `transformers`
- Approximately 40% lower memory consumption
- Output is a 160 MB adapter rather than 14 GB of full weights

	Vanilla	Unsloth
7B on 48 GB GPU	does not fit	fits in 4-bit
Speed	baseline	~2× faster
Memory	baseline	~40% less
Output artifact	14 GB	160 MB adapter

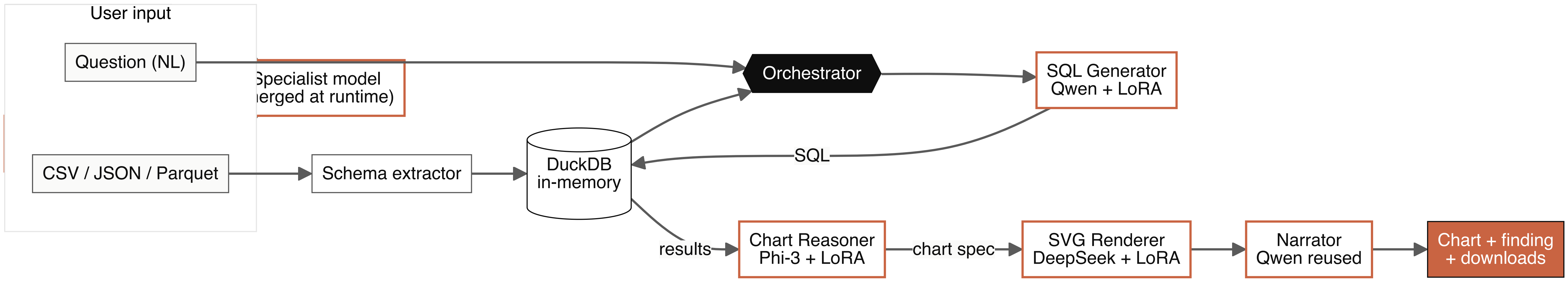
Three independent fine-tunes, one role each

Model	Base	Dataset	Hardware	Time	Final loss	Cost
SQL Generator	Qwen2.5-Coder-7B	text-to-sql-mix-v2 (672k)	L40S 48 GB	13.5 h	0.2658	~$24
Chart Reasoner	Phi-3-Mini 4k	chart-reasoning-mix-v1 (75k)	Colab / HF Jobs	~3 h	~0.31	~$3
SVG Renderer	DeepSeek-Coder 1.3B	svg-chart-render-v1 (25k)	Colab T4	~2 h	~0.40	~$1

Common training recipe

- QLoRA, r=16, α=32, 4-bit base
- 1 epoch, no early stopping
- TRL `packing=True` for the SQL Generator (4× speedup, 21 h → 13.5 h)
- Outputs: small adapters (22–161 MB), not full models

The system uses LoRA adapters, not full models



Adapter	Base model	Size
sql-generator-qwen25-coder-7b-lora	Qwen 2.5 Coder 7B	161 MB
chart-reasoner-phi3-mini-adapter-only	Phi-3 Mini 4k	38 MB
svg-renderer-deepseek-coder-1.3b-lora	DeepSeek Coder 1.3B	22 MB

The combined adapter weights total 221 MB. Base models remain public; only the task-specific adjustments are stored separately.

End-to-end query flow

Per query: four model invocations and a DuckDB execution, with a typical end-to-end latency of 5 to 8 seconds on a warm GPU. All adapters are loaded once at module level on a half-H200 via Hugging Face ZeroGPU.

When a generated SQL query fails to execute, the orchestrator retries up to three times with the error message added as context.

PHASE 6 — DEPLOYMENT AND COST

Total compute cost: approximately $30

Stage	Compute	Cost
SQL Generator training	HF Jobs L40S, 13.5 h	~$24
Chart Reasoner training	Colab / HF Jobs	~$3
SVG Renderer training	Colab / HF Jobs	~$1
GPT-4.1-nano dataset distillation	OpenAI Batch API	~$2.50
Inference hosting	HF Spaces ZeroGPU	$0
Total		**~$30**

Why the cost is low

- Unsloth QLoRA reduces training memory and time
- Sequence packing further reduces training time
- HF Spaces ZeroGPU provides free on-demand GPU for inference
- Only adapters are stored, not full model weights

DEMONSTRATION

Live system demo

Summary and future work

What we delivered

- Three open-source datasets on Hugging Face
- Three QLoRA adapters trained on those datasets
- A working multi-model agent that answers questions about uploaded data
- Total compute cost approximately $30

Limitations and future work

- Quantitative evaluation on Spider, WikiSQL, BIRD
- Multi-turn conversational memory
- Anomaly detection on uploaded data
- Statistical summary at ingestion

Repository: github.com/DanielRegaladoUMiami/sql-agent-llmops